

Einleitung & QA-Strategie

Bei unserem rundenbasierten Multiplayer-Spiel *Phantom Hunt* treten drei Geister gegen einen Menschen an. Während der Mensch versucht, zu entkommen, wollen die Geister ihn erschrecken. Das Spiel läuft über vier Runden, wobei die Rolle des Menschen rotiert, sodass jeder Spieler genau einmal in dieser Position spielt.

Die Serverarchitektur basiert auf einem *Thread-per-Client*-Modell mit TCP-Sockets. Jedem Client wird ein eigener Thread zugewiesen. Spielzustände und Nicknames werden zentral verwaltet. Der Fokus in der Qualitätssicherung liegt entsprechend auf der zentralen Scoring-Logik sowie der Netzwerkstabilität.

Orientiert am FCM-Modell verfolgt die Qualitätssicherung das Ziel, das System zuverlässig wartbar und testbar zu gestalten. Zur Messung wurden konkrete Qualitätsziele definiert. Die Testabdeckung in den kritischen Kernpaketen (game und protocol) wird auf 85-90% angehoben. Die zyklomatische Komplexität wird auf $V(G) \leq 4.0$ begrenzt. Fehler werden präventiv durch strukturierten Code vermieden und analytisch durch automatisierte sowie manuelle Tests nachgewiesen.

Konstruktives Qualitätsmanagement

Das konstruktive Qualitätsmanagement zielt darauf ab, Fehler durch geeignete Massnahmen bereits im Entwicklungsprozess zu verhindern.

Architektur & sicherer Code

Zur Vermeidung von Abstürzen durch nicht geschlossene Netzwerkverbindungen wird bei der Socket-Kommunikation konsequent auf try-with-resources-Blöcke gesetzt. Dadurch werden Ressourcen auch im Fehlerfall automatisch freigegeben.

Wartbarkeit & Zyklomatische Komplexität

Die zyklomatische Komplexität $v(G)$ wird mit dem Plugin *MetricsReloaded* überwacht. Als Grenzwert gilt $v(G) \leq 4.0$ über das gesamte Projekt, da niedrige Werte eine klare Methodenverantwortung (*Single Responsibility*) sowie eine vollständige Testabdeckung mit wenigen Unit-Tests sicherstellen. Komplexe Ausnahmen, etwa das switch-Statement im Protokoll-Parser, wurden gezielt isoliert. Der abschliessende Scan zeigt über alle Architekturschichten einen Durchschnittswert zwischen **1.57** und **2.34** und liegt damit deutlich unterhalb des Grenzwerts von **4.0**.

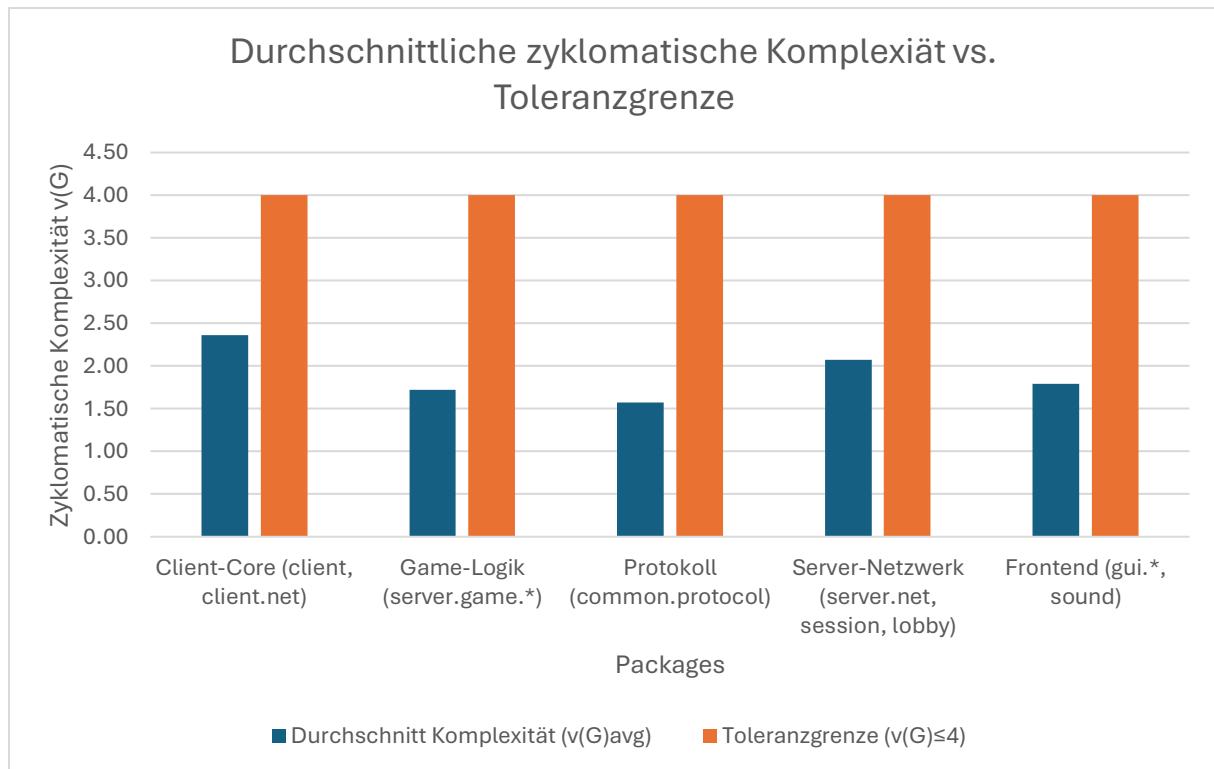


Abbildung 1: Komplexität pro Architekturschicht (MS6)

Statische Code-Analyse & Priorisierung

Aufgrund der parallelen Thread-Verarbeitung wurde der Code mittels statischer Analyse in IntelliJ IDEA auf bekannte Fehlermuster und Sicherheitsrisiken untersucht. Der initiale Scan lieferte über **4000** Warnungen, die sich bei näherer Betrachtung überwiegend als nicht relevante Hinweise erwiesen, etwa Rechtschreibprüfungen in Kommentaren oder Warnungen aus externen Frontend-Dateien.

Die Analyse fokussierte sich daher auf die architekturkritischen Java-Warnungen. Drei Kernprobleme wurden identifiziert und behoben:

1. **Sicherheit (Supply-Chain-Vulnerability):** Der Scanner meldete für die Logging-Bibliothek `log4j-core` (Version 2.25.3) eine bekannte Sicherheitslücke (CVE). In der `build.gradle` wurde umgehend ein Patch-Level-Upgrade auf eine sichere Version durchgeführt.
2. **Performance:** In der Klasse `GameModel` (Methode `getWinner()`) wurde eine ineffiziente String-Konkatenation mittels `+=` in einer Schleife festgestellt. Da dies zu erhöhtem Speicherverbrauch führt, wurde die Logik auf einen ressourcenschonenden `StringBuilder` umgestellt.
3. **Laufzeitsicherheit (Bugs):** Im `GameHandler` wurde auf ein `Optional` zugegriffen, ohne vorherige Prüfung auf einen vorhandenen Wert. Durch Ergänzung einer `.isPresent()`-Prüfung wurden potenzielle `NoSuchElementException`-Abstürze des Server-Threads behoben.

Analytisches Qualitätsmanagement: Testabdeckung

Das analytische Qualitätsmanagement dient der Überprüfung und dem Nachweis, dass das System sowohl unter Normal- als auch unter Extrembedingungen stabil und fehlerfrei operiert.

Testabdeckung (Unit-Tests)

Die funktionale Korrektheit der Kernlogik wurde mittels JUnit-Tests verifiziert und die Code Abdeckung über JaCoCo gemessen. Der strategische Fokus liegt dabei auf dem Backend (Server-Logik und Protokoll), da der Server im Multiplayer-Betrieb die *"Single Source of Truth"* darstellt und Ausfälle das gesamte System beeinträchtigen würden.

Durch systematisches Test-Driven Development konnte die Abdeckung im Kern-Paket `common.protocol` auf **99%** und in der zentralen Spiellogik (`server.game.*`) auf **84%** gesteigert werden. Die Zielvorgabe von **85-90%** ist in den kritischen Bereichen de facto erfüllt. Im Bereich Client-Core wurde das Ziel von **80%** mit einer Abdeckung von **31%** deutlich verfehlt. Diese Priorisierung ist darauf zurückzuführen, dass die Client-Netzwerk-Klassen stark mit asynchronen I/O-Streams und dem JavaFX-UI-Thread (`Platform.runLater`) gekoppelt sind, was automatisierte Tests hochgradig instabil macht. Die QA-Ressourcen wurden daher bewusst in das manuelle End-to-End-Testing umgeleitet, was einen deutlich höheren *Return on Investment* für die Stabilität des Endprodukts lieferte.

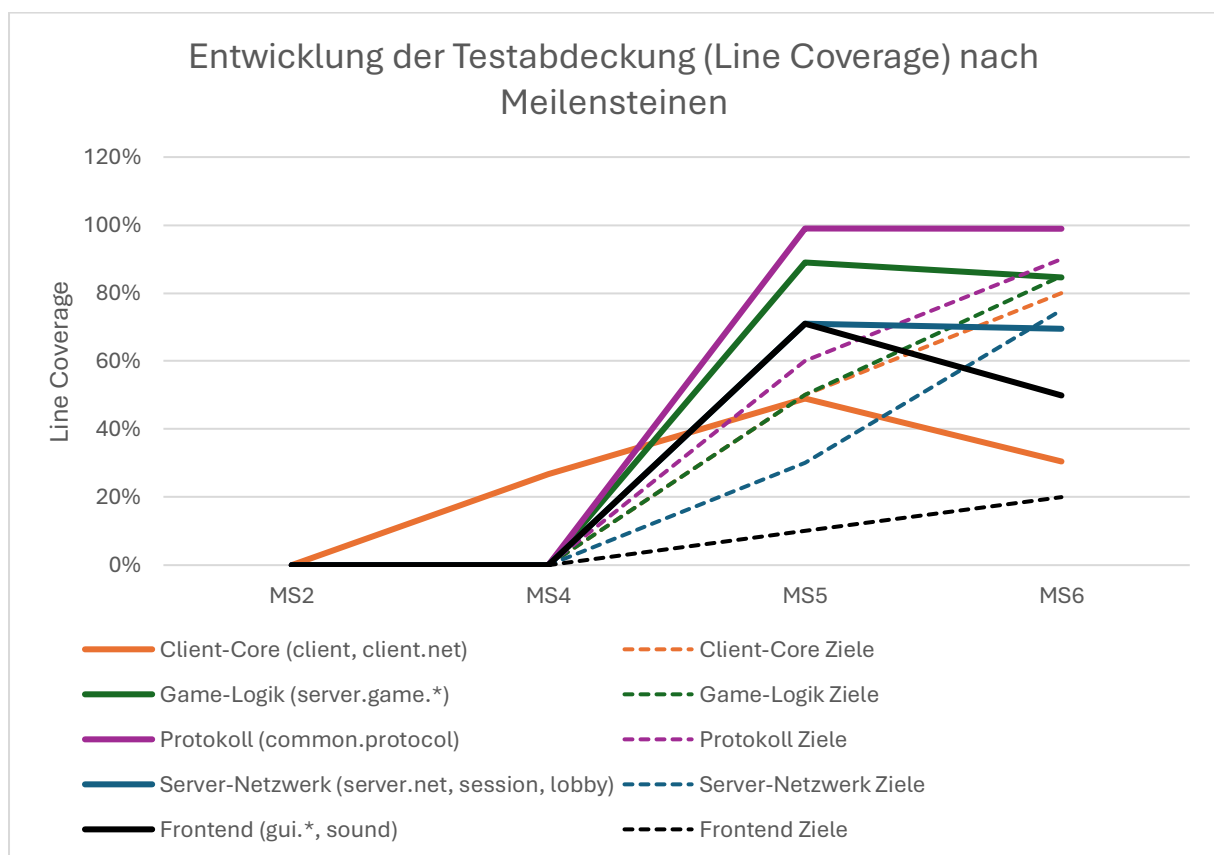


Abbildung 2: Testabdeckung pro Architekturschicht (MS4–MS6)

Systemstabilität & Concurrency (Thread-Safety)

Da die Architektur jedem verbundenen Client einen eigenen Thread zuweist, ist die Vermeidung von Nebenläufigkeitsfehlern essenziell. Zur Evaluierung des Systemverhaltens unter extremer Last wurde ein spezifischer Stresstest (`RegistryConcurrencyTest`) implementiert. Mittels eines `CountDownLatch` wurden **100** simulierte Clients dazu gebracht, auf die exakt gleiche Millisekunde denselben Nicknamen auf dem Server zu registrieren. Der erfolgreiche Testverlauf beweist, dass `ConcurrentHashMap` sowie die `synchronized`-Blöcke threadsicher sind und `Race Conditions` zuverlässig ausgeschlossen werden.

Ergänzend validieren unsere Black-Box-Tests, dass fehlerhafte oder manipulierte Protokoll-Strings vom System korrekt abgefangen und verworfen werden, ohne die Stabilität des Server-Threads zu beeinträchtigen.

Performance-Analyse & Profiling

Mithilfe des Java Flight Recorders (JFR) wurde das Laufzeitverhalten auf verschiedenen Hardware-Architekturen (Apple M1 Pro vs. Intel Linux) profiliert. Die Messungen belegen, dass das System nicht CPU-bound ist. Der Server verbraucht im Spielbetrieb nur minimale CPU-Ressourcen, und die Garbage Collection (G1) arbeitet mit Pausenzeiten von durchgehend unter **13 ms** äusserst effizient.

Die Analyse zeigte zudem, dass der Server unnötig Ressourcen allozierte, da er im Hintergrund JavaFX-Klassen geladen wurden. Diese architektonische Schwachstelle wurde behoben, sodass der Server ressourcenschonend und ohne User-Interface-Abhängigkeiten betrieben werden kann (siehe Lessons Learned).

End-to-End (E2E) Systemtest

Zur Validierung des realen Zusammenspiels der isoliert getesteten Komponenten wurde ein vollständiges E2E-Playtesting durchgeführt. Alle kritischen Systemzustände wurden vom Programm korrekt und gemäss Spezifikationen verarbeitet:

Test-ID	Test-Szenario (Prozessschritt)	Erwartetes Systemverhalten (Entscheidungsweg)	Resultat
E2E-01	4 Clients verbinden sich und registrieren Nicknamen.	Jeder Client erhält ein WELCOME-Paket, die Registry listet 4 eindeutige Namen.	[Pass]
E2E-02	Host erstellt Lobby, 3 weitere treten bei (MKL & CHECKIN).	Lobby-State wechselt auf "Voll" (4/4). Start-Button wird nur beim Host sichtbar.	[Pass]
E2E-03	Spielstart & Rollenzuweisung.	GameHandler instanziiert korrekt. 1 Human, 3 Phantoms werden über GSU-Pakete korrekt synchronisiert.	[Pass]

E2E-04	Phantom berührt Human (Kollision).	Runde endet sofort. Punkte werden berechnet, Rollen rotieren für die nächste Runde.	[Pass]
E2E-05	Simulierter Verbindungsabbruch (Ein Client wird hart geschlossen).	Server erkennt Disconnect. Laufendes Match wird abgebrochen (GamePhase.ABORTED), Spieler landen in der Lobby.	[Pass]

Abbildung 3: E2E Systemtest

Fazit & Lessons Learned

Der systematische QA-Prozess hat massgeblich dazu beigetragen, das System von einem instabilen Prototyp in eine robuste Multiplayer-Anwendung zu überführen. Die zentralen Erkenntnisse (Lessons Learned) lassen sich wie folgt zusammenfassen:

- **Architektonische Erkenntnisse ("Headless Server"):** Unser JFR-Profiling hat aufgedeckt, dass UI-Klassen im Server verwendet wurden (`javafx.scene.image.Image` in der `MapLogic`), nur um die Kachelgrösse der Map auszulesen. Nach Entkopplung dieser Logik läuft der Server ressourcenschonend auf Linux-Maschinen ohne Display ("headless"). Die Erkenntnis daraus: Backend und Frontend müssen auf Klassenebene strikt getrennt bleiben.
- **Prozessuale Erkenntnisse (Agilität vor Bürokratie):** Im Verlauf von Meilenstein 4 wurde der Einsatz einer formellen RACI-Matrix für die Aufgabenverteilung in Betracht gezogen. Es zeigte sich jedoch, dass ein solches Framework für die gegebene Teamgrösse administrativen Aufwand erzeugt. Die Verantwortlichkeiten liessen sich durch direkte, agile Kommunikation deutlich effizienter regeln. Wichtige QA-Ziele wurden dadurch schneller und ohne Verantwortungsdiffusion erreicht.
- **Methodische Erkenntnisse (Triage-Kompetenz bei Code-Analysen):** Die statische Code-Analyse liefert initial über 4000 Warnungen. Dabei zeigt sich, dass QA nicht das blinde Abarbeiten aller Hinweise bedeutet. Die Fähigkeit zur «Triage», also das Herausfiltern irrelevanter Warnungen (Frontend-Bibliotheken, Rechtschreibung) und die Fokussierung auf echte Risiken (Security-CVEs, Performance-Bugs), ist für ein effizientes Qualitätsmanagement essenziell.
- **Test-Erkenntnisse (Die Grenzen von isolierten Unit-Tests):** Unit-Tests mit isolierten Stubs (wie `FakeSocket` oder `FakeClientHandler`) eignen sich hervorragend, um die reine Spiellogik zu testen. Diese Tests sind jedoch blind gegenüber Multi-Threading-Problemen. Bei einer *Thread-per-Client* Architektur sind echte Concurrency-Stresstests (wie unser `RegistryConcurrencyTest` mit 100 parallelen Threads) zwingend notwendig, um Race Conditions aufzudecken, die in sequenziellen Unit-Tests nicht auftreten würden.

- **Test-Erkenntnisse (Return on Investment bei UI- und Async-Tests):** Für den Client-Core wurde initial eine Testabdeckung von 80% angestrebt. Es zeigte sich jedoch, dass automatisierte Tests für asynchronen Netzwerk-Code, der tief mit dem UI-Thread gekoppelt ist, hochgradig instabil sind. Die zentrale prozessuale Erkenntnis daraus: Bei stark gekoppelten UI- und Async-Komponenten ist es effizienter, Coverage-Ziele bewusst zurückzustellen und die QA-Ressourcen in striktes manuelles E2E-Playtesting zu investieren.
- **Performance-Erkenntnisse (Overhead durch Text-Protokolle):** Obwohl unser System nicht CPU-bound ist, zeigte das Profiling, dass ständiges String-Parsing (insbesondere die wiederholte Nutzung von Regex und `.split()` für die Spielzustands-Updates) relativ viel Leistung beansprucht. Für zukünftige Projekte ist festzuhalten, dass Serialisierung und Deserialisierung in *Hot Loops* (wie einem Game-Tick) durch striktere Formate oder den durchgehenden Einsatz von `StringBuilder` effizienter gestaltet werden sollte.